

Problem 0

```
In [1]: import numpy as np
import pandas as pd
import requests
import os
import io
import zipfile
import psycpg
from sqlalchemy import create_engine
import dotenv
dotenv.load_dotenv()
```

Out[1]: True

```
In [2]: # I did it once, which is why I commented out the bottom two lines, so that
url = 'https://databank.worldbank.org/data/download/ESG_CSV.zip'
r = requests.get(url)
# z = zipfile.ZipFile(io.BytesIO(r.content))
# z.extractall()
```

```
In [3]: # I did it once, which is why I commented out the bottom two lines, so that
url = 'https://v-dem.net/media/datasets/V-Dem-CY-Core_csv_v13.zip'
r = requests.get(url)
# z = zipfile.ZipFile(io.BytesIO(r.content))
# z.extractall()
```

```
In [4]: vdem = pd.read_csv('../data/V-Dem-CY-Core-v13.csv')
countrydata = pd.read_csv('../data/ESGCountry.csv')
wb = pd.read_csv('../data/ESGCSV.csv')
```

Problem 1

```
In [5]: vdem = vdem[['country_text_id', 'country_name', 'year', 'v2x_polyarchy']]
vdem = vdem.query("year >= 1960")
vdem = vdem.rename(columns={'country_text_id': "country_code", "country_name":
vdem = vdem.sort_values(by=['country_code', "year"], ascending=True)
```

Problem 2

```
In [6]: countrydata = countrydata[['Country Code', 'Table Name', 'Long Name', 'Curre
countrydata = countrydata.rename(columns={
    "Country Code": "country_code",
    "Table Name": "country_name_wb",
    "Long Name": "country_longname",
    "Currency Unit": "currency_unit",
    "Region": "region",
```

```
    "Income Group": "income_group"
})

noncountries = ["Arab World", "Central Europe and the Baltics", "Caribbean sm
"East Asia & Pacific (excluding high income)", "Early-demogra
"Europe & Central Asia (excluding high income)", "Europe & Ce
"European Union", "Fragile and conflict affected situations",
"Heavily indebted poor countries (HIPC)", "IBRD only", "IDA &
"IDA total", "IDA blend", "IDA only",
"Latin America & Caribbean (excluding high income)", "Latin A
"Low income", "Lower middle income", "Low & middle income",
"Late-demographic dividend", "Middle East & North Africa", "Mi
"Middle East & North Africa (excluding high income)", "North
"Other small states", "Pre-demographic dividend", "Pacific isl
"Post-demographic dividend", "Sub-Saharan Africa (excluding h
"Small states", "East Asia & Pacific (IDA & IBRD)", "Europe &
"Latin America & Caribbean (IDA & IBRD)", "Middle East & Nort
"South Asia (IDA & IBRD)", "Sub-Saharan Africa (IDA & IBRD)",

countrydata = countrydata.query("country_name_wb not in @noncountries")
countrydata
```

Out [6]:

	country_code	country_name_wb	country_longname	currency_unit	region	income
0	AFG	Afghanistan	Islamic State of Afghanistan	Afghan afghani	South Asia	
1	AGO	Angola	People's Republic of Angola	Angolan kwanza	Sub-Saharan Africa	Low
2	ALB	Albania	Republic of Albania	Albanian lek	Europe & Central Asia	Upper
3	AND	Andorra	Principality of Andorra	Euro	Europe & Central Asia	High
5	ARE	United Arab Emirates	United Arab Emirates	U.A.E. dirham	Middle East & North Africa	High
...	...	...	...	...	...	...
234	WSM	Samoa	Independent State of Samoa	Samoaan tala	East Asia & Pacific	Low
235	YEM	Yemen, Rep.	Republic of Yemen	Yemeni rial	Middle East & North Africa	
236	ZAF	South Africa	Republic of South Africa	South African rand	Sub-Saharan Africa	Upper
237	ZMB	Zambia	Republic of Zambia	New Zambian kwacha	Sub-Saharan Africa	Low
238	ZWE	Zimbabwe	Republic of Zimbabwe	Zimbabwean Dollar	Sub-Saharan Africa	Low

193 rows x 6 columns

Problem 3

```
In [7]: # I looked at the following stackoverflow post for the iteration through the
# https://stackoverflow.com/questions/75622174/select-dataframe-columns-that

columns = [c for c in wb.columns if c.startswith('19') or c.startswith('20')]
wb = wb[columns]
wb = wb.rename(columns={
```

```

    "Country Code": "country_code",
    "Country Name": "country_name_wb",
    "Indicator Code": "feature"
})
noncountries.remove("World")
wb = wb.query("country_name_wb not in @noncountries")
wb

```

Out [7]:

	country_name_wb	country_code	feature	1960	1961	19
3195	World	WLD	EG.CFT.ACCS.ZS	NaN	NaN	N
3196	World	WLD	EG.ELC.ACCS.ZS	NaN	NaN	N
3197	World	WLD	NY.ADJ.DRES.GN.ZS	NaN	NaN	N
3198	World	WLD	NY.ADJ.DFOR.GN.ZS	NaN	NaN	N
3199	World	WLD	AG.LND.AGRI.ZS	NaN	35.879317	35.952
...	...	...	...	...	...	
16964	Zimbabwe	ZWE	ER.PTD.TOTL.ZS	NaN	NaN	N
16965	Zimbabwe	ZWE	AG.LND.FRLS.HA	NaN	NaN	N
16966	Zimbabwe	ZWE	SL.UEM.TOTL.ZS	NaN	NaN	N
16967	Zimbabwe	ZWE	SP.UWT.TFRT	NaN	NaN	N
16968	Zimbabwe	ZWE	VA.EST	NaN	NaN	N

13774 rows × 67 columns

```

In [8]: replace_map = {
    'AG.LND.FRLS.HA': 'tree_cover_loss_hectares',
    'EN.CLC.CSTP.ZS': 'coastal_protection',
    'EN.CLC.HDDY.XD': 'heating_degree_days',
    'EN.H20.BDYS.ZS': 'proportion_water_good_quality',
    'EN.LND.LTMP.DC': 'land_surface_temperature',
    'ER.H20.FWST.ZS': 'level_of_water_stress',
    'SD.ESR.PERF.XQ': 'economic_social_rights_score',
    'AG.LND.AGRI.ZS': 'agricultural_land',
    'AG.LND.FRST.ZS': 'forest_area',
    'AG.PRD.FOOD.XD': 'food_production_index',
    'CC.EST': 'control_of_corruption',
    'EG.CFT.ACCS.ZS': 'access_to_clean_fuels_and_technologies_for_cooking',
    'EG.EGY.PRIM.PP.KD': 'energy_intensity_level_of_primary_energy',
    'EG.ELC.ACCS.ZS': 'access_to_electricity',
    'EG.ELC.COAL.ZS': 'electricity_production_from_coal_sources',
    'EG.ELC.RNEW.ZS': 'renewable_electricity_output',
    'EG.FEC.RNEW.ZS': 'renewable_energy_consumption',
    'EG.IMP.CONS.ZS': 'energy_imports',
    'EG.USE.COMM.FO.ZS': 'fossil_fuel_energy_consumption',
    'EG.USE.PCAP.KG.OE': 'energy_use',
    'EN.ATM.CO2E.PC': 'co2_emissions',
    'EN.ATM.METH.PC': 'methane_emissions',
    'EN.ATM.NOXE.PC': 'nitrous_oxide_emissions',

```

```

'EN.ATM.PM25.MC.M3': 'pm2_5_air_pollution',
'EN.CLC.CDDY.XD': 'cooling_degree_days',
'EN.CLC.GHGR.MT.CE': 'ghg_net_emissions',
'EN.CLC.HEAT.XD': 'heat_index_35',
'EN.CLC.MDAT.ZS': 'droughts',
'EN.CLC.PRCP.XD': 'maximum_5-day_rainfall',
'EN.CLC.SPEI.XD': 'mean_drought_index',
'EN.MAM.THRD.NO': 'mammal_species',
'EN.POP.DNST': 'population_density',
'ER.H2O.FWTL.ZS': 'annual_freshwater_withdrawals',
'ER.PTD.TOTL.ZS': 'terrestrial_and_marine_protected_areas',
'GB.XPD.RSDV.GD.ZS': 'research_and_development_expenditure',
'GE.EST': 'government_effectiveness',
'IC.BUS.EASE.XQ': 'ease_of_doing_business_rank',
'IC.LGL.CRED.XQ': 'strength_of_legal_rights_index',
'IP.JRN.ARTC.SC': 'scientific_and_technical_journal_articles',
'IP.PAT.RESD': 'patent_applications',
'IT.NET.USER.ZS': 'individuals_using_the_internet',
'NV.AGR.TOTL.ZS': 'agriculture',
'NY.ADJ.DFOR.GN.ZS': 'net_forest_depletion',
'NY.ADJ.DRES.GN.ZS': 'natural_resources_depletion',
'NY.GDP.MKTP.KD.ZG': 'gdp_growth',
'PV.EST': 'political_stability_and_absence_of_violence',
'RL.EST': 'rule_of_law',
'RQ.EST': 'regulatory_quality',
'SE.ADT.LITR.ZS': 'literacy_rate',
'SE.ENR.PRSC.FM.ZS': 'gross_school_enrollment',
'SE.PRM.ENRR': 'primary_school_enrollment',
'SE.XPD.TOTL.GB.ZS': 'government_expenditure_on_education',
'SG.GEN.PARL.ZS': 'proportion_of_seats_held_by_women_in_national_parliament',
'SH.DTH.COMM.ZS': 'cause_of_death',
'SH.DYN.MORT': 'mortality_rate',
'SH.H2O.SMDW.ZS': 'people_using_safely_managed_drinking_water_services',
'SH.MED.BEDS.ZS': 'hospital_beds',
'SH.STA.OWAD.ZS': 'prevalence_of_overweight',
'SH.STA.SMSS.ZS': 'people_using_safely_managed_sanitation_services',
'SI.DST.FRST.20': 'income_share_held_by_lowest_20pct',
'SI.POV.GINI': 'gini_index',
'SI.POV.NAHC': 'poverty_headcount_ratio_at_national_poverty_lines',
'SI.SPR.PCAP.ZG': 'annualized_average_growth_rate_in_per_capita_real_survey',
'SL.TLF.0714.ZS': 'children_in_employment',
'SL.TLF.ACTI.ZS': 'labor_force_participation_rate',
'SL.TLF.CACT.FM.ZS': 'ratio_of_female_to_male_labor_force_participation_rat',
'SL.UEM.TOTL.ZS': 'unemployment',
'SM.POP.NETM': 'net_migration',
'SN.ITK.DEFC.ZS': 'prevalence_of_undernourishment',
'SP.DYN.LE00.IN': 'life_expectancy_at_birth',
'SP.DYN.TFRT.IN': 'fertility_rate',
'SP.POP.65UP.TO.ZS': 'population_ages_65_and_above',
'SP.UWT.TFRT': 'unmet_need_for_contraception',
'VA.EST': 'voice_and_accountability'}

```

```

wb = wb.replace({"feature":replace_map})
wb

```

Out [8]:

	country_name_wb	country_code	featu
3195	World	WLD	access_to_clean_fuels_and_technologies_for_coc
3196	World	WLD	access_to_electrici
3197	World	WLD	natural_resources_depletio
3198	World	WLD	net_forest_depletio
3199	World	WLD	agricultural_lai
...	...	...	
16964	Zimbabwe	ZWE	terrestrial_and_marine_protected_are
16965	Zimbabwe	ZWE	tree_cover_loss_hectar
16966	Zimbabwe	ZWE	unemployme
16967	Zimbabwe	ZWE	unmet_need_for_contraceptio
16968	Zimbabwe	ZWE	voice_and_accountabili

13774 rows × 67 columns

Problem 4

a and b (I sort of combined them)

```
In [9]: # mainly use the pandas documentation for help with this one (which was not
# https://pandas.pydata.org/docs/reference/api/pandas.melt.html
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.pivot.html
index=[c for c in wb.columns if c.startswith('19') or c.startswith('20')]
wb = pd.melt(wb, id_vars=["country_name_wb", "country_code", "feature"], val
wb = pd.pivot(wb, values = "value", index = ['country_name_wb', 'country_cod
```

```
In [10]: # used this stack overflow to help with this question because I wasn't sure
# https://stackoverflow.com/questions/20490274/how-to-reset-index-in-a-panda
wb = wb.reset_index() # needed to do this to change the index back to numeri
wb
```

```
Out[10]:
```

feature	country_name_wb	country_code	variable	access_to_clean_fuels_and_technol
0	Afghanistan	AFG	1960	
1	Afghanistan	AFG	1961	
2	Afghanistan	AFG	1962	
3	Afghanistan	AFG	1963	
4	Afghanistan	AFG	1964	
...	...	...	...	...
12411	Zimbabwe	ZWE	2019	
12412	Zimbabwe	ZWE	2020	
12413	Zimbabwe	ZWE	2021	
12414	Zimbabwe	ZWE	2022	
12415	Zimbabwe	ZWE	2023	

12416 rows x 74 columns

```
In [11]: wb = wb.rename(columns={"variable": "year"})
```

C

```
In [12]: # I mainly use the pandas docs for this
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.astype.html
wb['year'] = wb['year'].astype("int")
wb = wb.query("year <= 2022")
wb
```

```
Out[12]:
```

feature	country_name_wb	country_code	year	access_to_clean_fuels_and_technologi
0	Afghanistan	AFG	1960	
1	Afghanistan	AFG	1961	
2	Afghanistan	AFG	1962	
3	Afghanistan	AFG	1963	
4	Afghanistan	AFG	1964	
...	...	...	...	...
12410	Zimbabwe	ZWE	2018	
12411	Zimbabwe	ZWE	2019	
12412	Zimbabwe	ZWE	2020	
12413	Zimbabwe	ZWE	2021	
12414	Zimbabwe	ZWE	2022	

12222 rows x 74 columns

d

```
In [13]: world_dataframe = wb.query("country_name_wb == 'World'")
wb = wb.query("country_name_wb != 'World'")
world_dataframe = world_dataframe.drop(["country_name_wb", "country_code"],

In [14]: world_dataframe = world_dataframe.rename(columns={"feature": "world_feature",
                                                         "value": "world_value"})

In [15]: vdem
```

```
Out[15]:
```

	country_code	country_name_vdem	year	democracy
5433	AFG	Afghanistan	1960	0.080
5434	AFG	Afghanistan	1961	0.083
5435	AFG	Afghanistan	1962	0.082
5436	AFG	Afghanistan	1963	0.085
5437	AFG	Afghanistan	1964	0.137
...	...	...	...	...
26151	ZZB	Zanzibar	2018	0.268
26152	ZZB	Zanzibar	2019	0.266
26153	ZZB	Zanzibar	2020	0.258
26154	ZZB	Zanzibar	2021	0.276
26155	ZZB	Zanzibar	2022	0.284

10550 rows x 4 columns

Problem 5

a

The `wb` dataframe is slightly larger than the `vdem` dataframe. For this reason, I believe that it is likely to be a close one to one match or a many to one (from `vdem` to `wb`) match for these two dataframes joining together. I believe the reason for this is that the data is very similar, with the `country_code` and `year` columns likely matching on a one to one basis. There are likely similar records in each dataframe for these combinations of `country_code` and `year`, thus I think that it makes sense that it will be a close one to one match. For lengths, see the code cell directly below.

```
In [16]: wb_length = wb.shape[0]
vdem_length = vdem.shape[0]

print("vdem_length:", vdem_length)
print("wb_length:", wb_length)
print("combined", vdem_length + wb_length)
```

```
vdem_length: 10550
wb_length: 12159
combined 22709
```

b

```
In [17]: vdem['year'] = vdem['year'].astype("int")
merged_df = pd.merge(vdem, wb, on=['country_code', 'year'], how='outer', ind
```

C

```
In [18]: merged_df.value_counts("matched")
```

```
Out[18]: matched
both      10148
right_only 2011
left_only  402
Name: count, dtype: int64
```

```
In [19]: # wb but not vdem
step_one = merged_df.query("matched == 'right_only'")
max_and_min_right = step_one.groupby(["country_code", "country_name_wb"]).ag
```

```
In [20]: # vdem but not wb
step_two = merged_df.query("matched == 'left_only'")
max_and_min_left = step_two.groupby(["country_code", "country_name_vdem"]).a
```

```
In [21]: max_and_min_left
```

```
Out[21]:
```

			year	
			max	min
country_code		country_name_vdem		
DDR	German Democratic Republic	1990	1960	
HKG	Hong Kong	2022	1960	
PSE	Palestine/West Bank	2022	1967	
PSG	Palestine/Gaza	2022	1960	
SML	Somaliland	2022	1991	
TWN	Taiwan	2022	1960	
VDR	Republic of Vietnam	1975	1960	
XKX	Kosovo	2022	1999	
YMD	South Yemen	1990	1960	
ZZB	Zanzibar	2022	1960	

```
In [22]: max_and_min_right
```

Out[22]:

country_code	country_name_wb	year	
		max	min
AND	Andorra	2022	1960
ARE	United Arab Emirates	1970	1960
ARM	Armenia	1989	1960
ATG	Antigua and Barbuda	2022	1960
AZE	Azerbaijan	1989	1960
BGD	Bangladesh	1970	1960
BHS	Bahamas, The	2022	1960
BIH	Bosnia and Herzegovina	1991	1960
BLR	Belarus	1989	1960
BLZ	Belize	2022	1960
BRN	Brunei Darussalam	2022	1960
CMR	Cameroon	1960	1960
DMA	Dominica	2022	1960
EST	Estonia	1989	1960
FSM	Micronesia, Fed. Sts.	2022	1960
GEO	Georgia	1989	1960
GRD	Grenada	2022	1960
HRV	Croatia	1990	1960
KAZ	Kazakhstan	1989	1960
KGZ	Kyrgyz Republic	1989	1960
KIR	Kiribati	2022	1960
KNA	St. Kitts and Nevis	2022	1960
LCA	St. Lucia	2022	1960
LIE	Liechtenstein	2022	1960
LTU	Lithuania	1989	1960
LVA	Latvia	1989	1960
MCO	Monaco	2022	1960
MDA	Moldova	1989	1960
MHL	Marshall Islands	2022	1960
MKD	North Macedonia	1990	1960

country_code	country_name_wb	year	
		max	min
MNE	Montenegro	1997	1960
NRU	Nauru	2022	1960
PLW	Palau	2022	1960
SMR	San Marino	2022	1960
SSD	South Sudan	2010	1960
SVK	Slovak Republic	1992	1960
SVN	Slovenia	1988	1960
TJK	Tajikistan	1989	1960
TKM	Turkmenistan	1989	1960
TON	Tonga	2022	1960
TUV	Tuvalu	2022	1960
UKR	Ukraine	1989	1960
UZB	Uzbekistan	1989	1960
VCT	St. Vincent and the Grenadines	2022	1960
WSM	Samoa	2022	1960

d

For this first question, I do not see any rows in either of the `left_only` or `right_only` tables that are unmatched based on spelling errors. There are many more unmatched from the `wb` table than there are for the `vdem` table.

From a technical perspective, the reason that some of the columns did not match is because of differences in the `country_code` and `year` columns. Perhaps not all `country_codes` that are supposed to match exist in the other table.

For the reason why records in `vdem` may not match with `wb` is likely because the records are not technically countries. As an example, take Hong Kong. Hong Kong is a ["Special Administrative Region"](#) in the People's Republic of China. Thus, `vdem` recognizes but perhaps not `wb`. Another reason is that it is a country that is broken up into regions like "Palestine/West Bank" and "Palestine/Gaza".

I think that for the reason why records in `wb` may not match with `vdem` is likely because of changes in a countries internal structure. For instance, [Azerbaijan](#) declared independence from the Soviet Union in the 1990s. Therefore, it's possible that the

vdem data lumped in Azerbaijan as a country in the Soviet Union. Furthermore, Bosnia and Herzegovina weren't countries before 1992, because they were a part of [Yugoslavia](#). So I think that the reason why records in wb may match with vdem is essentially because of country changes due to war, independence, etc.

e

```
In [23]: # dropping unmatched
merged_df = merged_df.query("matched == 'both']").drop("matched", axis=1)
```

Problem 6

```
In [24]: print(wb.columns)
```

```

Index(['country_name_wb', 'country_code', 'year',
      'access_to_clean_fuels_and_technologies_for_cooking',
      'access_to_electricity', 'agricultural_land', 'agriculture',
      'annual_freshwater_withdrawals',
      'annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_or_income',
      'cause_of_death', 'children_in_employment', 'co2_emissions',
      'coastal_protection', 'control_of_corruption', 'cooling_degree_days',
      'economic_social_rights_score',
      'electricity_production_from_coal_sources', 'energy_imports',
      'energy_intensity_level_of_primary_energy', 'energy_use',
      'fertility_rate', 'food_production_index', 'forest_area',
      'fossil_fuel_energy_consumption', 'gdp_growth', 'ghg_net_emissions',
      'gini_index', 'government_effectiveness',
      'government_expenditure_on_education', 'gross_school_enrollment',
      'heat_index_35', 'heating_degree_days', 'hospital_beds',
      'income_share_held_by_lowest_20pct', 'individuals_using_the_internet',
      'labor_force_participation_rate', 'land_surface_temperature',
      'level_of_water_stress', 'life_expectancy_at_birth', 'literacy_rate',
      'mammal_species', 'mean_drought_index', 'methane_emissions',
      'mortality_rate', 'natural_resources_depletion', 'net_forest_depletion',
      'net_migration', 'nitrous_oxide_emissions', 'patent_applications',
      'people_using_safely_managed_drinking_water_services',
      'people_using_safely_managed_sanitation_services',
      'pm2_5_air_pollution', 'political_stability_and_absence_of_violence',
      'population_ages_65_and_above', 'population_density',
      'poverty_headcount_ratio_at_national_poverty_lines',
      'prevalence_of_overweight', 'prevalence_of_undernourishment',
      'primary_school_enrollment',
      'proportion_of_seats_held_by_women_in_national_parliaments',
      'proportion_water_good_quality',
      'ratio_of_female_to_male_labor_force_participation_rate',
      'regulatory_quality', 'renewable_electricity_output',
      'renewable_energy_consumption', 'research_and_development_expenditure',
      'rule_of_law', 'scientific_and_technical_journal_articles',
      'strength_of_legal_rights_index',
      'terrestrial_and_marine_protected_areas', 'tree_cover_loss_hectares',
      'unemployment', 'unmet_need_for_contraception',
      'voice_and_accountability'],
      dtype='object', name='feature')

```

```
In [25]: print(countrydata.columns)
```

```

Index(['country_code', 'country_name_wb', 'country_longname', 'currency_unit',
      'region', 'income_group'],
      dtype='object')

```

```
In [26]: print(merged_df.columns)
```

```

Index(['country_code', 'country_name_vdem', 'year', 'democracy',
      'country_name_wb', 'access_to_clean_fuels_and_technologies_for_cookin
g',
      'access_to_electricity', 'agricultural_land', 'agriculture',
      'annual_freshwater_withdrawals',
      'annualized_average_growth_rate_in_per_capita_real_survey_mean_consum
ption_or_income',
      'cause_of_death', 'children_in_employment', 'co2_emissions',
      'coastal_protection', 'control_of_corruption', 'cooling_degree_days',
      'economic_social_rights_score',
      'electricity_production_from_coal_sources', 'energy_imports',
      'energy_intensity_level_of_primary_energy', 'energy_use',
      'fertility_rate', 'food_production_index', 'forest_area',
      'fossil_fuel_energy_consumption', 'gdp_growth', 'ghg_net_emissions',
      'gini_index', 'government_effectiveness',
      'government_expenditure_on_education', 'gross_school_enrollment',
      'heat_index_35', 'heating_degree_days', 'hospital_beds',
      'income_share_held_by_lowest_20pct', 'individuals_using_the_interne
t',
      'labor_force_participation_rate', 'land_surface_temperature',
      'level_of_water_stress', 'life_expectancy_at_birth', 'literacy_rate',
      'mammal_species', 'mean_drought_index', 'methane_emissions',
      'mortality_rate', 'natural_resources_depletion', 'net_forest_depletio
n',
      'net_migration', 'nitrous_oxide_emissions', 'patent_applications',
      'people_using_safely_managed_drinking_water_services',
      'people_using_safely_managed_sanitation_services',
      'pm2_5_air_pollution', 'political_stability_and_absence_of_violence',
      'population_ages_65_and_above', 'population_density',
      'poverty_headcount_ratio_at_national_poverty_lines',
      'prevalence_of_overweight', 'prevalence_of_undernourishment',
      'primary_school_enrollment',
      'proportion_of_seats_held_by_women_in_national_parliaments',
      'proportion_water_good_quality',
      'ratio_of_female_to_male_labor_force_participation_rate',
      'regulatory_quality', 'renewable_electricity_output',
      'renewable_energy_consumption', 'research_and_development_expenditur
e',
      'rule_of_law', 'scientific_and_technical_journal_articles',
      'strength_of_legal_rights_index',
      'terrestrial_and_marine_protected_areas', 'tree_cover_loss_hectares',
      'unemployment', 'unmet_need_for_contraception',
      'voice_and_accountability'],
      dtype='object')

```

a

In order to have a third normal form database, we need the following properties:

First normal

1. every table has a primary key
2. the values in each cell must be atomic
3. no repeating groups

Second normal 4. first normal 5. no columns are functionally dependent on only part of the primary key

Third normal: 5. first and second normal 6. no columns are functionally dependent on columns outside of the primary key

So, I think for this reason, these tables are likely almost in third normal form. First normal form is pretty easily satisfied. Second and third are a little weird. In our merged_df the columns are equally reliant on the country_name_wb as the country_name_vdem. Also all the features are repeated across both the different databases. I would just use one big dataframe for all of the features, based on the primary key of country_code and year. So, I don't think this is quite third normal form, but almost there. It violates the second normal form functional independence principle.

b

```
In [27]: def connect_to_postgres(password, user='postgres', host='localhost', port='5
        dbserver = psycopg.connect(user=user, password=password, host=host,
        dbserver.autocommit = True
        if recreate:
            # cursor is the location for writing code to run on the server
            cursor = dbserver.cursor()
            cursor.execute("DROP DATABASE IF EXISTS cardib")
            cursor.execute("CREATE DATABASE cardib")
        engine = create_engine(f"postgresql+psycopg://{user}:{password}@{hos
        return dbserver, engine
```

```
In [28]: pw = os.getenv("POSTGRES_PASSWORD")
        dbserver, engine = connect_to_postgres(password=pw)

        countrydata.to_sql("countrydata", con=engine, index=False, chunksize=1000, i
        wb.to_sql("wb", con=engine, index=False, chunksize=1000, if_exists="replace"
        merged_df.to_sql("timeseries", con=engine, index=False, chunksize=1000, if_e
```

Out[28]: -11

C

```
In [29]: def dbml_helper(data):
        dt = data.dtypes.reset_index().rename({0: 'dtype'}, axis=1)
        replace_map = {'object': 'varchar',
            'int64': 'int',
            'float64': 'float'}
        dt['dtype'] = dt['dtype'].replace(replace_map)
        return dt.to_string(index=False, header=False)
```

dbml file:

Project CARDIB { database_type: 'PostgreSQL' Note: "" # Lab3 CARDIB database assignment Data was collected from World Bank ESG data project and the University of Notre Dame. "" } Table countrydata as C { country_code varchar [pk] country_name_wb varchar [pk] country_longname varchar currency_unit varchar region varchar income_group varchar note: "country data baseline" } Table wb as W { country_name_wb varchar [pk] country_code varchar [pk] year int [pk] access_to_clean_fuels_and_technologies_for_cooking float access_to_electricity float agricultural_land float agriculture float annual_freshwater_withdrawals float annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_or_income float cause_of_death float children_in_employment float co2_emissions float coastal_protection float control_of_corruption float cooling_degree_days float economic_social_rights_score float electricity_production_from_coal_sources float energy_imports float energy_intensity_level_of_primary_energy float energy_use float fertility_rate float food_production_index float forest_area float fossil_fuel_energy_consumption float gdp_growth float ghg_net_emissions float gini_index float government_effectiveness float government_expenditure_on_education float gross_school_enrollment float heat_index_35 float heating_degree_days float hospital_beds float income_share_held_by_lowest_20pct float individuals_using_the_internet float labor_force_participation_rate float land_surface_temperature float level_of_water_stress float life_expectancy_at_birth float literacy_rate float mammal_species float mean_drought_index float methane_emissions float mortality_rate float natural_resources_depletion float net_forest_depletion float net_migration float nitrous_oxide_emissions float patent_applications float people_using_safely_managed_drinking_water_services float people_using_safely_managed_sanitation_services float pm2_5_air_pollution float political_stability_and_absence_of_violence float population_ages_65_and_above float population_density float poverty_headcount_ratio_at_national_poverty_lines float prevalence_of_overweight float prevalence_of_undernourishment float primary_school_enrollment float proportion_of_seats_held_by_women_in_national_parliaments float proportion_water_good_quality float ratio_of_female_to_male_labor_force_participation_rate float regulatory_quality float renewable_electricity_output float renewable_energy_consumption float research_and_development_expenditure float rule_of_law float scientific_and_technical_journal_articles float strength_of_legal_rights_index float terrestrial_and_marine_protected_areas float tree_cover_loss_hectares float unemployment float unmet_need_for_contraception float voice_and_accountability float note: "Data from the World Bank" } Table merged as M { country_code varchar [pk] country_name_vdem varchar [pk] year int [pk] democracy float country_name_wb varchar [pk] access_to_clean_fuels_and_technologies_for_cooking float access_to_electricity float agricultural_land float agriculture float annual_freshwater_withdrawals float annualized_average_growth_rate_in_per_capita_real_survey_mean_consumption_or_income float cause_of_death float children_in_employment float co2_emissions float coastal_protection float control_of_corruption float cooling_degree_days float economic_social_rights_score float electricity_production_from_coal_sources float energy_imports float energy_intensity_level_of_primary_energy float energy_use float fertility_rate float food_production_index float forest_area float fossil_fuel_energy_consumption float gdp_growth float ghg_net_emissions float gini_index float government_effectiveness float government_expenditure_on_education float gross_school_enrollment float heat_index_35 float heating_degree_days float hospital_beds float income_share_held_by_lowest_20pct float individuals_using_the_internet float labor_force_participation_rate float land_surface_temperature float level_of_water_stress float life_expectancy_at_birth float literacy_rate float mammal_species float mean_drought_index float methane_emissions float mortality_rate float natural_resources_depletion float net_forest_depletion float net_migration float nitrous_oxide_emissions float patent_applications float people_using_safely_managed_drinking_water_services float people_using_safely_managed_sanitation_services float pm2_5_air_pollution float political_stability_and_absence_of_violence float population_ages_65_and_above float population_density float poverty_headcount_ratio_at_national_poverty_lines float prevalence_of_overweight float prevalence_of_undernourishment float primary_school_enrollment float proportion_of_seats_held_by_women_in_national_parliaments float proportion_water_good_quality float ratio_of_female_to_male_labor_force_participation_rate float regulatory_quality float renewable_electricity_output float renewable_energy_consumption float research_and_development_expenditure float rule_of_law float scientific_and_technical_journal_articles float strength_of_legal_rights_index float terrestrial_and_marine_protected_areas float tree_cover_loss_hectares float unemployment float unmet_need_for_contraception float voice_and_accountability float note: "Data merging vdem and wb data" } Ref: C.country_code < M.country_code Ref: C.country_code < W.country_code Ref: C.country_name_wb < M.country_name_wb Ref: C.country_name_vdem < M.country_name_vdem Ref: M.country_name_wb - W.country_name_wb Ref: M.year - W.year

URL

<https://dbdocs.io/j-miskill/CARDIB>

Problem 7

a

```
In [30]: query_one = """
SELECT a.country_name_wb AS country, a.democracy
FROM timeseries AS a
LEFT JOIN countrydata as b
ON a.country_code=b.country_code
WHERE a.year=2022
ORDER BY a.democracy desc
"""
pd.read_sql_query(query_one, con=engine)
```

```
Out[30]:
```

	country	democracy
0	Denmark	0.916
1	Norway	0.899
2	Sweden	0.899
3	Switzerland	0.898
4	Estonia	0.893
...	...	...
167	Korea, Dem. People's Rep.	0.087
168	Afghanistan	0.082
169	China	0.075
170	Eritrea	0.073
171	Saudi Arabia	0.016

172 rows × 2 columns

b

```
In [31]: # I'm not sure this is correct, given the question, but it is correct according to the question
query_two = """
SELECT a.year, b.life_expectancy_at_birth, a.life_expectancy_at_birth
FROM timeseries AS a
INNER JOIN wb AS b
ON a.country_code=b.country_code AND a.year=b.year
WHERE a.country_code='CHL'
ORDER BY a.year
"""
pd.read_sql_query(query_two, con=engine)
```

```
Out[31]:
```

	year	life_expectancy_at_birth	life_expectancy_at_birth
0	1960	57.015	57.015
1	1961	57.537	57.537
2	1962	57.771	57.771
3	1963	57.150	57.150
4	1964	58.738	58.738
...	...	...	...
58	2018	80.133	80.133
59	2019	80.326	80.326
60	2020	79.377	79.377
61	2021	NaN	NaN
62	2022	NaN	NaN

63 rows x 3 columns

C

```
In [32]: query_three = """
SELECT b.region,
SUM(a.co2_emissions) AS co2_emissions
FROM timeseries AS a
INNER JOIN countrydata AS b
ON a.country_code=b.country_code
WHERE a.year=2020
GROUP BY b.region
ORDER BY co2_emissions DESC;
"""
pd.read_sql_query(query_three, con=engine)
```

```
Out[32]:
```

	region	co2_emissions
0	Europe & Central Asia	252.500224
1	Middle East & North Africa	169.621034
2	East Asia & Pacific	89.022518
3	Latin America & Caribbean	60.383198
4	Sub-Saharan Africa	39.999251
5	North America	26.632203
6	South Asia	8.793069

d

```
In [33]: query_four = """
SELECT a.country_code, (b.democracy-a.democracy) AS dem_score
FROM (
    SELECT country_code, democracy
    FROM timeseries
    WHERE year=1960
) AS a
INNER JOIN (
    SELECT country_code, democracy
    FROM timeseries
    WHERE year=2022
) AS b
ON a.country_code=b.country_code
ORDER BY dem_score DESC
LIMIT 10;
"""
pd.read_sql(query_four, con=engine, )
```

```
Out[33]:
```

	country_code	dem_score
0	ESP	0.803
1	CPV	0.727
2	PRT	0.718
3	VUT	0.691
4	TLS	0.677
5	CZE	0.657
6	STP	0.654
7	SYC	0.631
8	MWI	0.602
9	NAM	0.573

e

```
In [34]: query_five = """
SELECT currency_unit, count(*) as num_countries
FROM countrydata
GROUP BY currency_unit
ORDER BY num_countries DESC
"""
pd.read_sql(query_five, con=engine)
```

```
Out[34]:
```

	currency_unit	num_countries
0	Euro	24
1	West African CFA franc	8
2	U.S. dollar	7
3	East Caribbean dollar	6
4	Central African CFA franc	6
...	...	...
138	Lao kip	1
139	Czech koruna	1
140	Iraqi dinar	1
141	New Zambian kwacha	1
142	Sierra Leonean leone	1

143 rows x 2 columns

```
In [35]: query_six = """
SELECT a.income_group, AVG(b.gini_index) AS avg_gini_score
      FROM countrydata AS a
      INNER join wb AS b
      ON a.country_code=b.country_code
WHERE b.year=2022
GROUP BY a.income_group
      """
pd.read_sql(query_six, con=engine)
```

```
Out[35]:
```

	income_group	avg_gini_score
0	High income	41.800000
1	Lower middle income	35.271429
2	Low income	32.000000
3	Upper middle income	42.300000
4	None	NaN

g

```
In [36]: # https://stackoverflow.com/questions/14290857/sql-select-where-field-contain
# it seems like there aren't any columns that contain the word Republic or D
# I even tried to search using Pandas and I did not find anything
republic = "Republic"
democratic = "Democratic"

query_seven = f"""
```

```
SELECT a.republic, a democratic, avg(b.democracy) AS dem_score
FROM (
    SELECT country_code, country_name_wb, country_name_vdem, year,
           CASE WHEN country_name_wb LIKE '%s{republic}%s' or country_name_vdem
           CASE WHEN country_name_wb LIKE '%s{democratic}%s' or country_name_vd
           FROM timeseries
) AS a
LEFT JOIN timeseries AS b
ON a.country_code=b.country_code
WHERE b.year=2022
GROUP BY a.republic, a democratic
""""

pd.read_sql(query_seven, con=engine)
```

```
Out[36]:
```

	republic	democratic	dem_score
0	0	0	0.502582

```
In [ ]:
```